

Project map

Every folder, every file and every table, with a two-line explanation of each

What the project does: you ask a mutual fund database a question in plain English; it works out which of 20 tables to use and how to join them, writes safe read-only SQL, and returns the table, an explanation, and proof of every check.

Where things live: the data is in Amazon RDS (PostgreSQL), the schema graph is in Amazon Neptune, the models are on Amazon Bedrock, and the application runs on your own machine. **79 files in 13 folders.**

The folders at a glance

Folder	What it holds	Files
(top level)	Settings, packages, the README	5
infra/aws/	Files the one-time AWS setup commands read	4
config/	Every rule and limit the pipeline follows	6
scripts/	A helper that prepares the RDS instance	1
dataplane/	Creates the synthetic fund data. Deleted when using a real database	14
adapters/	The only code that talks to RDS and to Neptune	2
metadata/	Teaches the system the database: build → review → publish	8
agent/	The 11-step workflow that answers a question	5
guardrails/	Safety checks between a question and the database	2
api/	The web server and the two web pages	4
evals/	Scores how well the system answers. Never read by the pipeline	8
tests/	One file per stage; a stage is done when its file passes	8
memory/ , jobs/	Empty, reserved for later stages	—

Top level

File	What it is
.env	Your real keys and addresses: AWS key pair, the Bedrock model, the guardrail, the two database URLs, the Neptune address. Never committed to Git
.env.example	The same setting names with blank values, safe to share. Copy it to .env and fill it in
.gitignore	Tells Git what to ignore: .env , the virtual environment, caches. This is what keeps your keys off GitHub
requirements.txt	Every Python package the project needs. Installed with <code>pip install -r requirements.txt</code>
pyproject.toml	Settings for the test runner and the code linter only. No packages are listed here
README.md	The stack, the setup order, and how to use it on a real database

infra/aws/ — files the AWS commands read

File	What it is
iam-policy.json	The permissions for the application's user: call Bedrock models, apply the guardrail, query Neptune, and nothing else
guardrail-content-policy.json	The guardrail's one filter: block prompt attacks, checking input only, at high strength
budget.json	A monthly spending limit, so nothing runs up a bill unnoticed
budget-notifications.json	Who to email, and at what percentage of that limit

config/ — the rules

These hold rules the pipeline reads. They never hold answers.

File	What it is
models.yaml	The embedding model (Titan) and its size, 1,024 numbers, plus the longest reply allowed. The chat model comes from .env
agent.yaml	The agent's limits: 2 SQL retries, a 15-second timeout, a 5,000-row cap, and a cost ceiling
metrics.yaml	The 8 approved business definitions, so "net flows" is calculated the same way every time
metadata.yaml	How links are proposed (95% of values must match), what gets embedded, and the longest join route
pii.yaml	The personal-data columns. Anything listed here is never sent to a model and never embedded
generator.yaml	The random seed, the data size, and the five-year window used when creating the test data

scripts/

File	What it is
rds_setup.py	Run once after creating the RDS instance. Creates the second database, agent_meta , and switches on pgvector

`dataplane/` — creates the synthetic fund data

Delete this whole folder when pointing the system at a real database.

`dataplane/schema/` — the empty tables

File	What it is
<code>01_fund_house.sql</code>	Creates 9 tables: categories, benchmarks, schemes, plans, managers, daily prices, monthly AUM
<code>02_market.sql</code>	Creates 6 tables: sectors, groups, issuers, securities, credit ratings, holdings
<code>03_investors.sql</code>	Creates 4 tables: distributors, investors, folios, SIPs
<code>04_transactions.sql</code>	Creates the transactions table, split into 60 monthly pieces so date filters read only the months they need
<code>apply_schema.py</code>	Runs those four files in order against RDS. Safe to re-run: each run rebuilds from empty

`dataplane/generator/` — 1.1 million rows

File	What it is
<code>run.py</code>	The one command. Runs everything below in order, then hands the rows to the loader
<code>common.py</code>	Shared helpers: reads the settings, works out trading days and month ends, holds the name and city lists
<code>patterns.py</code>	The six planted stories as fixed values: which bond, which scheme, which dates, how big
<code>fund_house.py</code>	Builds tables 1-8: schemes, plans, managers, and five years of daily prices
<code>market.py</code>	Builds tables 10-14: sectors, groups, issuers, securities and their credit ratings
<code>investors.py</code>	Builds tables 16-19: distributors, investors, their accounts and SIPs
<code>transactions.py</code>	Builds the transactions table. Checks each account's holdings before any redemption, so none goes below zero
<code>summaries.py</code>	Builds tables 9 and 15 by calculating them, so AUM and holdings agree exactly

`dataplane/load/`

File	What it is
<code>copy_loader.py</code>	Empties the 20 tables and loads them with PostgreSQL's fast COPY, in an order that respects the links

`adapters/` — the only code that talks to a database

File	What it is
<code>postgres.py</code>	Four jobs on RDS: read the catalog, read statistics, estimate a query's cost, and run a query read-only
<code>graph_neptune.py</code>	Talks to Neptune: writes the graph, reads its edges back, and signs every request with your AWS keys

`metadata/` — teaches the system the database

Runs in three steps: build → review → publish. Writes only to `agent_meta`.

File	What it is
<code>build.py</code>	Step 1. Reads the database, finds the links, and asks the model to draft a note for every table and column
<code>review.py</code>	Step 2. Prints every note and link for a person to read and approve. Nothing unapproved is ever used
<code>publish.py</code>	Step 3. Takes only approved items: embeds them into pgvector, and rebuilds the graph in Neptune
<code>links.py</code>	Finds links three ways: declared foreign keys, matching column names, and matching values. This is what finds the hidden links
<code>notes.py</code>	Decides what the model is shown for each table: columns, types and examples, never personal data
<code>retrieval.py</code>	The three lookups the agent uses: search tables, match a typed name to an exact value, and find a join route
<code>store.py</code>	Creates and opens the four <code>meta_</code> tables in <code>agent_meta</code>
<code>llm.py</code>	The only code in the project that calls Bedrock: chat replies and embeddings

`agent/` — the 11 steps that answer a question

File	What it is
<code>graph.py</code>	Wires the 11 steps together with their branches: blocked, retry, give up
<code>context.py</code>	Loads once and shares: the settings, the database connections, the graph edges, the approved notes
<code>steps/understand.py</code>	Steps 1-3. Checks the question for attacks, works out what is being asked, and matches names to exact values
<code>steps/build_query.py</code>	Steps 4-8. Picks tables, finds the joins in Neptune, writes the SQL, checks it, and runs it read-only
<code>steps/answer.py</code>	Steps 9-11. Summarises the result, writes the explanation, and remembers the turn for follow-ups

`guardrails/` — safety

File	What it is
input_guard.py	Sends the question to the Bedrock Guardrail before anything else, to catch attempts to override the AI
sql_checks.py	Reads the SQL's structure: one statement, SELECT only, known tables only, and a real date range on the big table

api/ — the server and the pages

File	What it is
main.py	The web endpoints: ask a question, fetch the graph, serve the two pages, report health
telemetry.py	Switches on tracing before the server starts, so each request and step can be seen afterwards
static/index.html	The demo page: the question box, the 11 steps, the graph, the answer, the SQL and its checks
static/graph.html	The graph page: all 20 tables, clickable, with hidden links shown differently

evals/ — how good are the answers?

Nothing here runs when a user asks a question. The pipeline never reads this folder, which is what makes its scores meaningful.

File	What it is
run_eval.py	The one file you run. Asks 18 questions, scores every answer, prints a scorecard, saves the run
questions/questions.yaml	The 18 questions, each with the tables, joins and words a correct answer must involve
metrics/retrieval.py	Marks the finding part: precision, recall, ranking, and whether the required joins were used
metrics/generation.py	Marks the answer: is every number in it actually in the rows, and does it mention what was asked
metrics/judge.py	Optional. Asks the model to score the wording out of 5, for things arithmetic cannot see
metrics/ragas_adapter.py	Optional. Translates the results for RAGAS, a well-known evaluation library
answer_key/hidden_links.yaml	The three links deliberately left undeclared, used to score how many were discovered
answer_key/planted_patterns.yaml	The six stories planted in the data, used to check the answers are right

tests/ — is it still working?

File	Checks	Count
test_rds_setup.py	Both databases reachable, on one instance, with pgvector working	7
test_bedrock.py	Every setting present, the chat model replies in JSON, embeddings are 1,024 numbers	5
test_guardrail.py	The guardrail is reachable, allows a normal question, and blocks an attack	3
test_neptune.py	The graph cluster answers, and a node can be written and read back	4
test_stage1_schema.py	20 tables, 60 monthly pieces, hidden links still undeclared	11
test_stage2_data.py	Gate 1: the data is consistent and all six stories are visible	13
test_stage5_metadata.py	Gate 2: notes approved, hidden links found, graph correct, search works	19
test_stage6_agent.py	Unsafe SQL refused, every demo question answered, attacks blocked	18
Total		80

The 20 tables in mf_data

1,132,667 rows, about 210 MB. A fictitious fund house with 40 schemes over five years.

#	Table	One row is	Rows
1	scheme_categories	One SEBI category, such as Large Cap Fund	36
2	benchmarks	One index a scheme is measured against	15
3	benchmark_values	One index on one trading day	19,560
4	schemes	One fund. Almost every question passes through this table	40
5	scheme_plans	One version of a fund: Direct or Regular, Growth or IDCW	150
6	fund_managers	One person who runs funds	25
7	scheme_manager_assignments	One manager on one scheme for one period	74
8	nav_history	The price of one plan on one day	189,482
9	scheme_aum_monthly	The money in one scheme at one month end	2,320
10	sectors	One industry	25
11	issuer_groups	One business group, such as Trident Group	60
12	issuers	One company, bank or government that issues securities	800
13	securities	One share, bond or short-term paper	1,500
14	credit_rating_history	One rating given to a bond on one date. Old-style column names	1,973
15	portfolio_holdings	One security held in one scheme at one month end	111,072
16	distributors	One agent or platform that sells schemes. Old-style column names	600
17	investors	One person who invests. Personal data, masked at creation	40,000
18	folios	One investor's account in one scheme	57,776
19	sip_registrations	One standing instruction to invest monthly or quarterly	24,000
20	transactions	One purchase, SIP payment, redemption or switch. 60 monthly pieces	683,159

The four tables in `agent_meta`

This is the agent's own database: what it knows **about** the fund data. None of it is fund data.

Table	One row is	Rows
meta_tables	One table's plain-English note, with what one row means and its status	20
meta_columns	One column's note, and whether it holds personal data	124
meta_edges	One link between two tables, how it was found, and whether a person approved it	22
meta_embeddings	One piece of text turned into 1,024 numbers, so similar meanings can be found	1,415

The graph in Neptune holds 20 table nodes and 22 join edges. It is rebuilt from `meta_edges` every time you publish, so that table stays the source of truth.

The three hidden links the system discovered by itself

Link	Why it is hard
scheme_aum_monthly.scheme_id → schemes.scheme_id	Same column name, but no foreign key declares it
credit_rating_history.sec_cd → securities.security_code	Completely different names. 780 values, all matching
transactions.arn_code → distributors.arn_no	Different names again. 574 values, all matching